M08 Review / Repair - Teaching RC2

OPEN ONLY AFTER SAVED ATTEMPT

Use surgically

Read only the competency you missed. Close this file and solve a changed retry. Reading review is not mastery.

P01 repair - Ingestion architecture: source -> landing -> staging

Ingestion is not merely reading a file. You need to preserve what arrived, know where transformation is allowed, and be able to replay without asking the source to recreate history.

Core repair rule: Source is where data originates: file drop, API, database or event stream.

Proof: quarantine -> preserve rejected evidence

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P02 repair - Defensive CSV ingestion

CSV looks simple, so ingestion bugs are often silent: embedded delimiters, missing columns, inconsistent date/number formats and extra fields.

Core repair rule: Use a real CSV parser rather than split(",").

Proof: source mutation -> none

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P03 repair - JSON and nested-record ingestion

APIs often return nested arrays/objects. Blind normalization can duplicate parent fields or lose the relationship between parent and child records.

Core repair rule: Inspect top-level type/keys and one complete sample before writing flattening code.

Proof: child rows -> reconciled

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P04 repair - Parquet ingestion and metadata inspection

Parquet stores typed columnar data and metadata that can make analytical reads more efficient and self-describing.

Core repair rule: Parquet is columnar and stores schema/types with the file.

Proof: raw source -> still preserved

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P05 repair - REST API contracts, status codes and timeouts

An API can return valid HTTP responses that still represent errors, partial results or a changed contract. Reliable ingestion treats the contract explicitly.

Core repair rule: A GET request can still fail with 4xx/5xx statuses; do not parse every body as success data.

Proof: business -> record validity after parsing

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P06 repair - API authentication, pagination and rate limits

Many real APIs deliberately return only one page and limit request rate. Missing pages silently creates incomplete data.

Core repair rule: Read credentials from environment/secret storage, not committed code.

Proof: secret in logs -> none

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P07 repair - Retries, backoff, jitter and transient failures

Retries can recover transient failures, but reckless retries can overload services, repeat non-idempotent actions and hide deterministic bugs.

Core repair rule: Retry selected transient failures such as timeouts/connection resets, 429 and some 5xx according to API semantics.

Proof: schema mismatch -> quarantine/repair

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P08 repair - Database extraction and safe query boundaries

Database sources can be huge and mutable. SELECT * without predicate/window can create expensive, inconsistent or unreproducible extracts.

Core repair rule: Connect with least required privileges; ingestion readers usually do not need schema ownership/write access.

Proof: source write -> none

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P09 repair - Historical batch landing and raw metadata

Backfills often involve many old files. If you overwrite by filename or process in accidental order, history and replay become unreliable.

Core repair rule: Keep source logical date/event period separate from ingestion processing time.

Proof: content hash -> identity/dedup evidence

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P10 repair - Incremental loads and high-water marks

Incremental loads reduce repeated work, but a badly chosen watermark can silently lose late rows or double boundary rows.

Core repair rule: Choose a stable monotonic source field when possible: immutable increasing ID or reliable updated_at + tie-breaker key.

Proof: failure -> old checkpoint retained

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P11 repair - CDC concepts and change-event semantics

Change Data Capture provides changes, not just snapshots. A consumer must understand operation type, key, ordering and replay behavior.

Core repair rule: CDC event typically identifies table/entity key, operation type and before/after or changed values plus ordering metadata.

Proof: duplicate event -> no duplicate business effect

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P12 repair - Schema drift, compatibility, time zones and quarantine

External producers change. If your parser silently accepts changed meaning, downstream data can become wrong without an obvious crash.

Core repair rule: Schema drift includes added/removed/renamed fields, type changes and semantic changes that may not be machine-detectable.

Proof: timestamp without defined zone -> unsafe

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P13 repair - Checkpoint/state management and replay safety

Process memory disappears, jobs crash and retries happen. State is part of the pipeline correctness model.

Core repair rule: State can be a watermark, cursor, CDC offset, processed content identity or combination.

Proof: after state commit -> start after committed state

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P14 repair - Idempotent landing and content identity

Retries and backfills frequently present the same content again. Idempotency means repeated execution reaches the same durable result, not "never retry".

Core repair rule: Content identity can use source object ID/version/ETag or a cryptographic hash when appropriate.

Proof: business key -> semantic record identity

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P15 repair - Late-arriving data, event time and processing time

Data can arrive minutes or days after the event occurred. Watermarks, partitions and reports are wrong if arrival time is mistaken for business event time.

Core repair rule: Event time describes when the business event happened; **processing/ingestion time** describes when your pipeline handled it.

Proof: lag -> how late was it?

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.

P16 repair - Ingestion observability: counts, latency and run metadata

Without run metadata, an empty target could mean no source data, failed extraction, filter bug or rejected batch. Observability reduces guesswork.

Core repair rule: Give every run a unique run_id and source/window identity.

Proof: checkpoint movement -> incremental progress

STOP - check your understanding

Close Review now. Can you solve a changed version without copying?

Answer in your own words before continuing.